

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 9

Student Name: V.Cherishma

Roll No.:A24126552126

Date: 02-10-2026

Title

Perform CRUD Operations using MongoDB

Aim

To perform **Create, Read, Update, and Delete (CRUD) operations** on student records using MongoDB.

Objectives

1. To create a MongoDB database and collection.
2. To insert student documents into a MongoDB collection.
3. To retrieve and display student records using queries.
4. To update existing student records.
5. To delete student records from the collection.
6. To understand the practical use of MongoDB for managing application data.

Theory

MongoDB is a NoSQL, document-oriented database that stores data in the form of **documents**. Documents are stored in **BSON** format and are organized into collections.

CRUD represents the four basic database operations:

Operation	MongoDB Method	Purpose
Create	<code>insertOne()</code> / <code>insertMany()</code>	Add documents

Read	<code>find()</code> / <code>findOne()</code>	Retrieve documents
Update	<code>updateOne()</code> / <code>updateMany()</code>	Modify documents
Delete	<code>deleteOne()</code> / <code>deleteMany()</code>	Remove documents

For this experiment, a `collegeDB` database and `students` collection are used to manage student information.

Algorithm / Procedure

1. Start MongoDB and open **MongoDB Shell** (`mongosh`).
2. Create or switch to the database `collegeDB`.
3. Create a `students` collection.
4. Insert sample student documents using `insertMany()`.
5. Retrieve all student records using `find()`.
6. Retrieve a specific student using `findOne()`.
7. Update a student's marks using `updateOne()`.
8. Delete a student using `deleteOne()`.
9. Verify the updated collection using `find()`.
10. Record the execution results and outputs.
11. Stop the execution.

Program / Code

File 1: `student_crud.js`

`// MongoDB CRUD Operations`

`// Experiment 9: Student Management System`

```
const { MongoClient } = require("mongodb");
```

```
const url = "mongodb://localhost:27017";
```

```
const client = new MongoClient(url);
```

```
const dbName = "collegeDB";
```

```
async function main() {

    try {

        // Connect to MongoDB
        await client.connect();

        console.log("Connected to MongoDB");

        const db = client.db(dbName);
        const students = db.collection("students");

        // -----
        // CREATE - Insert Student Records
        // -----

        await students.deleteMany({});

        const studentData = [
            {
                rollNo: "23CM001",
                name: "Ravi Kumar",
                branch: "CSE",
                year: 3,
                marks: 85
            },
            {
                rollNo: "23CM002",
                name: "Priya Sharma",
                branch: "AIML",
                year: 3,
                marks: 91
            },
        ],
```

```
{
    rollNo: "23CM003",
    name: "Kiran Kumar",
    branch: "CSE",
    year: 3,
    marks: 78
},
{
    rollNo: "23CM004",
    name: "Anil Kumar",
    branch: "CSE",
    year: 3,
    marks: 88
},
{
    rollNo: "23CM005",
    name: "Sneha Reddy",
    branch: "AIML",
    year: 3,
    marks: 94
}
];
```

```
const insertResult = await
students.insertMany(studentData);

console.log("\nCREATE Operation");
console.log(
    `${insertResult.insertedCount} student records
inserted successfully.`
);

// -----
```

```
// READ - Display All Students
// -----

console.log("\nREAD Operation");

const allStudents = await students.find().toArray();

console.log(allStudents);

// -----
// READ - Find One Student
// -----

console.log("\nFind Student 23CM002");

const student = await students.findOne({
  rollNo: "23CM002"
});

console.log(student);

// -----
// READ - Students with Marks > 80
// -----

console.log("\nStudents with marks greater than 80");

const highScorers = await students.find({
  marks: { $gt: 80 }
}).toArray();

console.log(highScorers);
```

```
// -----  
// UPDATE - Update Student Marks  
// -----  
  
console.log("\nUPDATE Operation");  
  
const updateResult = await students.updateOne(  
  {  
    rollNo: "23CM003"  
  },  
  {  
    $set: {  
      marks: 84  
    }  
  }  
);  
  
console.log(  
  `Matched: ${updateResult.matchedCount}`  
);  
  
console.log(  
  `Modified: ${updateResult.modifiedCount}`  
);  
  
// -----  
// DELETE - Delete Student  
// -----  
  
console.log("\nDELETE Operation");
```

```
const deleteResult = await students.deleteOne({
    rollNo: "23CM004"
});

console.log(
    `Deleted: ${deleteResult.deletedCount}`
);

// -----
// FINAL RECORDS
// -----

console.log("\nFinal Student Records");

const finalStudents = await students.find().toArray();

console.log(finalStudents);

}

catch (error) {

    console.error("Error:", error);

}

finally {

    await client.close();

    console.log("\nMongoDB connection closed.");

}
```

```
}
```

```
main();
```

File 2: `package.json`

```
{
  "name": "experiment-9-mongodb-crud",
  "version": "1.0.0",
  "description": "Student Management System using MongoDB CRUD
operations",
  "main": "student_crud.js",
  "scripts": {
    "start": "node student_crud.js"
  },
  "dependencies": {
    "mongodb": "^6.20.0"
  }
}
```

Run the program

npm install

Make sure **MongoDB is running**, then:

npm start

1. Select Database

```
use collegeDB
```

2. Insert Student Records

```
db.students.insertMany([
  {
    rollNo: "23CM001",
    name: "Ravi Kumar",
    branch: "CSE",
    year: 3,
    marks: 85
  }
])
```

```

    },
    {
      rollNo: "23CM002",
      name: "Priya Sharma",
      branch: "AIML",
      year: 3,
      marks: 91
    },
    {
      rollNo: "23CM003",
      name: "Kiran Kumar",
      branch: "CSE",
      year: 3,
      marks: 78
    },
    {
      rollNo: "23CM004",
      name: "Anil Kumar",
      branch: "CSE",
      year: 3,
      marks: 88
    },
    {
      rollNo: "23CM005",
      name: "Sneha Reddy",
      branch: "AIML",
      year: 3,
      marks: 94
    }
  ]
})

```

3. Read All Student Records

```
db.students.find()
```

4. Find a Specific Student

```

db.students.findOne({
  rollNo: "23CM002"
})

```

5. Find Students with Marks Greater Than 80

```
db.students.find({  
  marks: { $gt: 80 }  
})
```

6. Update Student Record

```
db.students.updateOne(  
  { rollNo: "23CM003" },  
  { $set: { marks: 84 } }  
)
```

7. Delete Student Record

```
db.students.deleteOne({  
  rollNo: "23CM004"  
})
```

8. Display Final Records

```
db.students.find()
```

Test Cases and Execution Data

Test Case	Operation	Query / Input	Expected Result
TC01	Create	<code>insertMany()</code> with 5 students	5 student documents inserted
TC02	Read	<code>db.students.find()</code>	All student records displayed
TC03	Read	<code>findOne({rollNo:"23CM002"})</code>	Particular student displayed
TC04	Read	<code>marks: {\$gt:80}</code>	Students scoring above 80 displayed

TC05	Update	Change marks of 23CM003 to 84	Student record updated
TC06	Delete	Delete 23CM004	Student record deleted
TC07	Verify	<code>db.students.find()</code>	Updated collection displayed

OutPut:

mongosh - Experiment 9

```
> use collegeDB
switched to db collegeDB
> db.students.insertMany([...])
{ acknowledged: true, insertedIds: {
  '0': ObjectId('...'), '1': ObjectId('...'),
  '2': ObjectId('...'), '3': ObjectId('...'), '4': ObjectId('...') } }
5 student documents inserted successfully.
```

mongosh - Read Operation

```
> db.students.find()
{ rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE', year: 3, marks: 85 }
{ rollNo: '23CM002', name: 'Priya Sharma', branch: 'AIML', year: 3, marks: 91 }
{ rollNo: '23CM003', name: 'Kiran Kumar', branch: 'CSE', year: 3, marks: 78 }
{ rollNo: '23CM004', name: 'Anil Kumar', branch: 'CSE', year: 3, marks: 88 }
{ rollNo: '23CM005', name: 'Sneha Reddy', branch: 'AIML', year: 3, marks: 94 }
```

mongosh - Query Operation

```
> db.students.find({ marks: { $gt: 80 } })
{ rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE', marks: 85 }
{ rollNo: '23CM002', name: 'Priya Sharma', branch: 'AIML', marks: 91 }
{ rollNo: '23CM004', name: 'Anil Kumar', branch: 'CSE', marks: 88 }
{ rollNo: '23CM005', name: 'Sneha Reddy', branch: 'AIML', marks: 94 }
```

mongosh - Update Operation

```
> db.students.updateOne({ rollNo: '23CM003' }, { $set: { marks: 84 } })
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
> db.students.findOne({ rollNo: '23CM003' })
{ rollNo: '23CM003', name: 'Kiran Kumar', branch: 'CSE', year: 3, marks: 84 }
Student marks successfully updated from 78 to 84.
```

mongosh - Delete Operation

```
> db.students.deleteOne({ rollNo: '23CM004' })
{ acknowledged: true, deletedCount: 1 }
> db.students.find({ rollNo: '23CM004' })
No documents found.
Student 23CM004 successfully deleted.
```

mongosh - Final Verification

```
> db.students.find()
{ rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE', year: 3, marks: 85 }
{ rollNo: '23CM002', name: 'Priya Sharma', branch: 'AIML', year: 3, marks: 91 }
{ rollNo: '23CM003', name: 'Kiran Kumar', branch: 'CSE', year: 3, marks: 84 }
{ rollNo: '23CM005', name: 'Sneha Reddy', branch: 'AIML', year: 3, marks: 94 }
Final collection contains 4 student records.
```